Sahad Rafiuzzaman

CS 580: Introduction to Artificial Intelligence

Dr. Lusi Li

March 2, 2025

Analysis Report for Homework Assignment 2

This report analyzes the performance of the Sudoku as a Constraint Satisfaction Problem (CSP) Solver. This homework assignment uses two solvers: Simple Backtracking and Smart Backtracking. This Sudoku project has four predefined difficulty levels: Easy, Medium, Hard, and Evil. The Smart Backtracking algorithm uses Minimum Remaining Value (MRV), Least Constraining Value (LCV), and Forward Checking (FC) to improve efficiency in solving the puzzle. The codes used to develop this assignment are based on the original csp.py codes provided by the AIMA GitHub library. This Sudoku solver uses a Graphical User Interface (GUI) developed using Tkinter.

The first step in this project was to define Sudoku as a CSP. Determining the variables, domains, and constraints were required to model the Sudoku. The variables are every empty cell within the 9x9 Sudoku grid. The domains are all possible real whole integers from 1 to 9. The constraints are to ensure that no two identical numbers appear in the same row, column, or 3x3 sub-grid. Defining the CSP variables, domains, and constraints allowed the solver to use search algorithms to solve each puzzle while not violating any of the rules of Sudoku.

I developed this Sudoku solver on a Tkinter-based GUI to help visualize the Sudoku grid better. Running the code will take the user to the welcome screen, where they can select the difficulty level of their choice. Figure 1 is a screenshot of the welcome screen. Once a difficulty is selected, the predefined Sudoku grid appears in a new window. The user can return to the welcome screen on this window by clicking back or using Simple Backtracking or Smart Backtracking to solve the puzzle. Figure 2 is a screenshot of an unsolved, easy Sudoku puzzle. Once an algorithm is selected, a message box will show the result. Figure 3 is a screenshot of a solved, easy Sudoku puzzle with the message box. The message box presents runtime, correct, incorrect, and total assignments for the respective Sudoku.

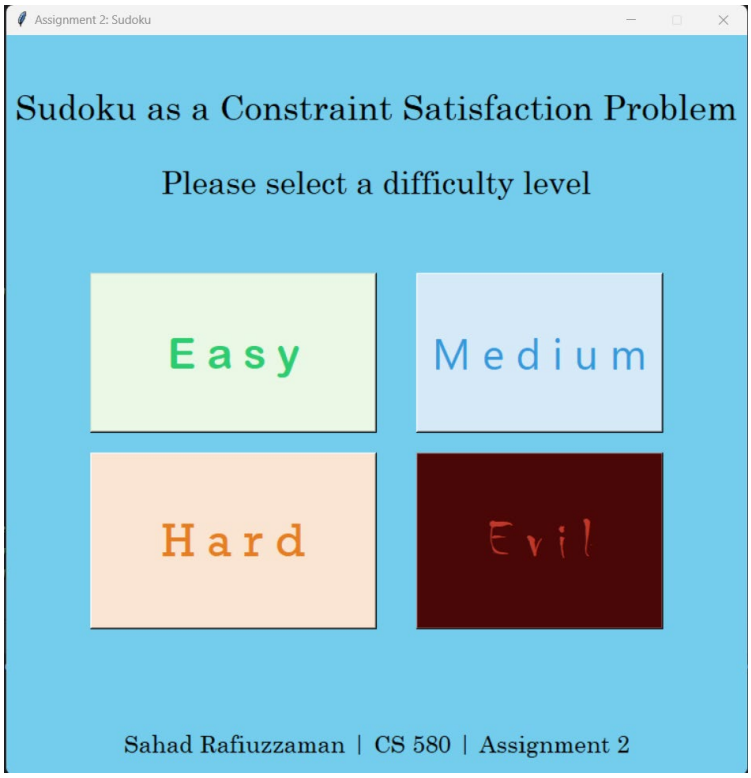

Figure 1: The welcome screen for the Sudoku as a CSP Solver.

Figure 2: An unsolved Easy Sudoku Puzzle.

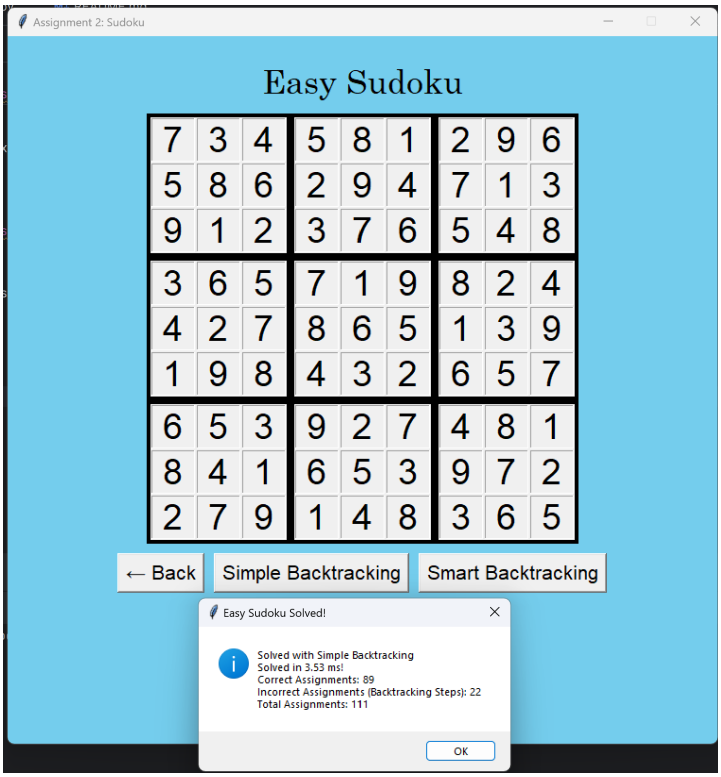

Figure 3: A solved Easy Sudoku Puzzle with the message box.

Simple Backtracking operates by finding an empty cell and trying to assign a value from the domain. It then verifies that the assignment doesn't violate any constraints. If the assignment is valid, it will move to the next empty cell and repeat its recursive process. If the assigned value is incorrect, it will backtrack and try a different value. It will continue until the board is solved. While Simple Backtracking will find a solution, if a solution exists, it becomes inefficient for puzzles with more complex difficulty levels. Since Simple Backtracking does not use heuristics to guide its search, it can lead to longer runtimes, as shown in Table 1.

The Smart Backtracking is built on Simple Backtracking and includes MRV, LCV, and FC. MRV picks an empty cell with the fewest possible values. LCV orders the selection of values to minimize future conflicts. Forward checking eliminates invalid values from neighboring cells when assigning a value. By integrating these heuristics, Smart Backtracking significantly has better runtimes for more complex puzzles than Simple Backtracking performance metrics. Unlike Simple Backtracking, Smart Backtracking does not randomly select the values, leading to faster solutions.

Table 1 shows the performance results for each Sudoku puzzle to evaluate both algorithms effectively. The runtime is the time taken to solve the puzzle in milliseconds. Correct assignments are the number of successful attempts to assign the proper value. Incorrect assignments are the number of failed attempts to assign the correct value, resulting in backtracking. Total assignments are the summation of correct and incorrect assignments.

Table 1: The Performance Results for each Sudoku Puzzle

Puzzle	Solver Type	Runtime (ms)	Correct Assignment	Incorrect Assignment	Total Assignment
Easy	Simple	3.51	89	22	111
Easy	Smart	4.5	81	18	99
Medium	Simple	8.51	88	30	118
Medium	Smart	4.51	82	31	113
Hard	Simple	151.45	98	35	133
Hard	Smart	14.51	84	48	132
Evil	Simple	267.01	89	35	124
Evil	Smart	6.51	83	48	131

The results show that Smart Backtracking is generally faster than Simple Backtracking, especially for more complex puzzles. For the Easy puzzle, Smart Backtracking had a slightly higher runtime due to the added heuristic overhead. Smart Backtracking significantly reduces runtime for the Hard and Evil puzzles while keeping assignments comparable. Incorrect assignments are higher in Smart Backtracking in Medium, Hard, and Evil level puzzles. In conclusion, Smart Backtracking performs better on more complex puzzles, reducing overall solving time. Meanwhile, Simple Backtracking can be effective for smaller grids but becomes inefficient for complex puzzles.